

SENTETİK EĞİTİM MATERYALİ

Modern C++

Temeller, Nesne Yönelimli Programlama, STL, Bellek Yönetimi ve İleri Konular

Doküman Kodu	SYN-CPP-ADV-001
Seviye	İleri / Uygulamalı
Amaç	Ders, katalog testi ve eğitim içeriği değerlendirmesi

Not: Bu doküman yapay olarak oluşturulmuş örnek eğitim içeriğidir. Gerçek bir kurumun resmi eğitim kaydı, onayı veya müfredatı değildir.

1. Eğitim Tanımı ve Öğrenme Çıktıları

Bu eğitim, C++ programlama dilini sıfırdan veya temel seviyeden öğrenen katılımcıları modern C++ geliştirme pratiklerine taşımak için hazırlanmıştır. Konular dil sözdiziminden proje yapısına, testten eşzamanlılığa kadar ilerler.

Hedef ortam: C++17 veya C++20 destekleyen bir derleyici; önerilen araçlar GCC, Clang veya MSVC. Editör/IDE olarak VS Code, Visual Studio veya CLion kullanılabilir.

Öğrenme çıktıları

- C++ temel sözdizimi, kontrol yapıları ve fonksiyonları kullanmak.
- Sınıf, kalıtım, kapsülleme ve polimorfizmle nesne yönelimli tasarım yapmak.
- STL container, iterator ve algoritmalarını etkin kullanmak.
- Pointer, referans ve RAIİ ile güvenli kaynak yönetimi uygulamak.
- Modern C++ özelliklerini (auto, lambda, smart pointer, move semantics vb.) kullanmak.
- Derleyici, debugger, test ve statik analiz araçlarıyla hata ayıklamak.
- CMake tabanlı çok dosyalı proje yapısı kurmak.
- Template, concurrency ve performans konularına giriş yapmak.

Modül	Süre
Temeller & Sözdizimi	5 saat
OOP	5 saat
STL	4 saat
Bellek & RAIİ	4 saat
Modern C++	4 saat
Debug & Test	3 saat
CMake & Proje Yapısı	3 saat
İleri Konular	4 saat

2. C++ Temelleri ve Sözdizimi

C++ kaynak kodu derlenerek makine koduna dönüştürülür. Tip sistemi güçlüdür ve performans kontrolü yüksektir. İlk aşamada değişkenler, koşullar, döngüler, fonksiyonlar ve temel I/O yapıları öğrenilir.

```
#include <iostream>
#include <string>

int main() {
    std::string isim;
    int puan{};

    std::cout << "Isim: ";
    std::getline(std::cin, isim);
    std::cout << "Puan: ";
    std::cin >> puan;

    if (puan >= 60) {
        std::cout << isim << " basarili.\n";
    } else {
        std::cout << isim << " gelistirmeli.\n";
    }

    return 0;
}
```

Kavram	Örnek
Temel tipler	int, double, bool, char
Standart metin	std::string
Koşul	if / else, switch
Döngü	for, while, range-based for
Fonksiyon	parametre, dönüş tipi, overload
Sabitlik	const, constexpr

Uygulama

Bir ürün listesindeki fiyatların ortalamasını, en yüksek ve en düşük değerini hesaplayan konsol uygulaması yazın. Girdi doğrulaması ekleyin.

3. Nesne Yönelimli Programlama

C++ sınıfları veri ve davranışı aynı yapı altında birleştirir. Kapsülleme, kalıtım ve polimorfizm büyük sistemlerin parçalara ayrılmasına yardımcı olur; ancak gereksiz kalıtım yerine çoğu zaman bileşim (composition) tercih edilir.

```
#include <iostream>
#include <memory>
#include <string>

class Sensor {
public:
    virtual ~Sensor() = default;
    virtual double read() const = 0;
    virtual std::string name() const = 0;
};

class TemperatureSensor final : public Sensor {
public:
    explicit TemperatureSensor(double value) : value_(value) {}
    double read() const override { return value_; }
    std::string name() const override { return "Temperature"; }
private:
    double value_;
};
```

- public / private / protected erişim belirleyicileri
- Constructor ve destructor
- Encapsulation: sınıf iç durumunu kontrollü arayüzle koruma
- Inheritance: uygun olduğu yerde “is-a” ilişkisi
- Virtual fonksiyonlar ve runtime polymorphism
- Composition: “has-a” ilişkisiyle daha esnek tasarım

Tasarım notu: Bir sınıfın tek bir sorumluluğu olması, test edilebilirlik ve bakım açısından genellikle daha sağlıklıdır.

4. Standart Template Library (STL)

STL, sık kullanılan veri yapıları ve algoritmalar için yüksek performanslı, jenerik bileşenler sunar. Container + iterator + algorithm üçlüsü modern C++ kodunun temel yapı taşlarındandır.

Bileşen	Kullanım
std::vector	Dinamik, ardışık veri dizisi
std::array	Sabit boyutlu dizi
std::map / unordered_map	Anahtar-değer eşleme
std::set	Tekil değer koleksiyonu
std::sort	Sıralama algoritması
std::find_if	Koşula uyan ilk elemanı bulma
std::accumulate	Toplama/katlama işlemleri

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> values{7, 2, 9, 4, 1};
    std::sort(values.begin(), values.end());

    auto it = std::find_if(values.begin(), values.end(),
        [](int x) { return x > 5; });

    if (it != values.end())
        std::cout << "İlk 5'ten büyük: " << *it << '\n';
}
```

Algoritmaları manuel döngüler yerine kullanmak niyeti daha açık ifade eder ve standart kütüphane optimizasyonlarından yararlanmayı sağlar.

5. Bellek Yönetimi, Pointer, Referans ve RAIİ

C++ geliştiricisinin en kritik sorumluluklarından biri kaynak yaşam süresidir. Modern C++ yaklaşımı çıplak new/delete kullanımını azaltır; kaynak sahipliğini nesnelerin yaşam süresine bağlayan RAIİ ve smart pointer yapıları tercih edilir.

Kavram	Öneri
Reference (T&)	Var olan nesneye null olmayan takma ad gerektiğinde
Pointer (T*)	Null olabilen veya adres semantiği gereken non-owning ilişkilerde
std::unique_ptr	Tek sahipli dinamik nesne
std::shared_ptr	Gerçekten ortak sahiplik gerektiğinde
std::weak_ptr	shared_ptr döngülerini kırmak için
RAIİ	Dosya, kilit, bellek gibi kaynakları nesne yaşam süresiyle yönetmek

```
#include <memory>

class Device {
public:
    void start() {}
};

int main() {
    auto device = std::make_unique<Device>();
    device->start();
} // device otomatik olarak serbest bırakılır
```

Kural: Sahiplik açıkça tanımlanamıyorsa tasarım yeniden düşünülmelidir. Smart pointer kullanmak tek başına iyi tasarım garantisi değildir.

6. Modern C++ (C++11/14/17/20)

Özellik	Amaç
auto	Türü derleyicinin çıkarması; özellikle iterator ve karmaşık dönüş tiplerinde okunabilirlik
range-based for	Koleksiyonları daha güvenli ve kısa dolaşma
lambda	Yerel, kısa fonksiyon nesneleri
move semantics	Gereksiz kopyaları azaltma ve kaynak transferi
std::optional	Bir değerin var/yok durumunu açıkça modelleme
std::variant	Tip güvenli alternatif değerler
structured bindings	Tuple/pair/struct ayrıştırmayı kolaylaştırma
concepts (C++20)	Template gereksinimlerini daha açık ifade etme

```
#include <algorithm>
#include <vector>

std::vector<int> data{1,2,3,4,5,6};
std::erase_if(data, [](int x) { return x % 2 == 0; }); // C++20
```

Move semantics özeti

Bir nesnenin pahalı kaynağı artık kaynak nesne tarafından kullanılmayacaksa, kopyalamak yerine taşıma yapılabilir. Standard container ve sınıflar bu mekanizmadan yoğun biçimde yararlanır.

7. Derleme, Derleyici Araçları ve Proje Yapısı

Tek dosyalı örneklerden gerçek projelere geçerken kaynak dosyaları, başlık dosyaları, bağımlılıklar ve derleme ayarları düzenli bir yapıda yönetilmelidir.

Araç	Rol
GCC / Clang / MSVC	Kaynak kodu derlemek
Compiler warnings	Potansiyel hata ve şüpheli kodu erken yakalamak
CMake	Platformlar arası build konfigürasyonu üretmek
Make / Ninja	Derleme adımlarını çalıştırmak
Package manager	vcpkg / Conan gibi araçlarla bağımlılık yönetmek

Örnek proje yapısı

```
project/
  CMakeLists.txt
  include/
    calculator.hpp
  src/
    calculator.cpp
    main.cpp
  tests/
    test_calculator.cpp
```

Basit CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)
project(TrainingApp LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
add_executable(training_app src/main.cpp src/calculator.cpp)
target_include_directories(training_app PRIVATE include)
```


8. Hata Ayıklama ve Test Teknikleri

- Derleyici uyarılarını yüksek seviyede açın (-Wall -Wextra veya MSVC /W4).
- Debugger ile breakpoint, step over/into, call stack ve watch pencerelerini kullanın.
- AddressSanitizer/UndefinedBehaviorSanitizer ile bellek ve UB sorunlarını yakalayın.
- Birim testlerini küçük, deterministik ve bağımsız tutun.
- Hata mesajlarını yalnızca bastırmak yerine kök nedeni analiz edin.

Mini test örneği (assert)

```
#include <cassert>

int add(int a, int b) { return a + b; }

int main() {
    assert(add(2, 3) == 5);
    assert(add(-1, 1) == 0);
}
```

Gerçek projelerde Catch2, GoogleTest veya doctest gibi test çatılarından biri kullanılabilir. CI ortamında testlerin her commit veya pull request ile otomatik çalıştırılması önerilir.

9. İleri Konular: Template, Eşzamanlılık ve Performans

Template ve jenerik programlama

```
template <typename T>
T clamp_value(T value, T low, T high) {
    if (value < low) return low;
    if (value > high) return high;
    return value;
}
```

Template mekanizması, aynı algoritmanın farklı tiplerle derleme zamanında yeniden kullanılmasını sağlar. İleri seviyede type traits, constexpr metaprogramlama ve concepts konuları devreye girer.

Eşzamanlılık

- std::thread ve std::jthread
- std::mutex ve lock_guard
- std::atomic
- std::future / std::async
- Data race ve deadlock riskleri

Performans yaklaşımı: Önce ölçün, sonra optimize edin. Profiling yapılmadan “daha hızlıdır” varsayımıyla karmaşık kod yazmak bakım maliyetini artırabilir.

10. Uygulama Projesi: Sensör İzleme Konsolu

Katılımcı, farklı sensör tiplerinden veri alan ve eşik ihlallerini raporlayan çok dosyalı bir C++ uygulaması geliştirecektir. Proje modern C++ ve test tekniklerini birlikte kullanmalıdır.

Gereksinim	Beklenen çözüm
En az iki sensör türü	Ortak interface + polymorphism
Kayıt koleksiyonu	std::vector veya uygun STL container
Sahiplik	std::unique_ptr tercihli açık yaşam süresi
Filtreleme / sıralama	STL algorithms + lambda
Yapı	include/src/tests klasörleri
Build	CMake
Test	En az 5 birim testi
Ek puan	std::jthread ile periyodik okuma veya üretici-tüketici örneği

Değerlendirme

- Doğruluk ve işlev: %35
- Kod kalitesi ve modern C++ kullanımı: %25
- Bellek/kaynak güvenliği: %15
- Test edilebilirlik: %15
- Build ve proje düzeni: %10

11. Kısa Ölçme ve Değerlendirme

1. Tek sahipli dinamik nesne için hangi yapı uygundur?

- A) `std::shared_ptr`
- B) `std::unique_ptr`
- C) raw new zorunludur
- D) `std::map`

2. Bir sınıfın taban sınıf arayüzünü çalışma zamanında farklı uygulaması hangi kavramdır?

- A) Polimorfizm
- B) Preprocessor
- C) Linking
- D) Casting

3. Çok platformlu C++ build konfigürasyonunda yaygın araç hangisidir?

- A) Power Query
- B) CMake
- C) VBA
- D) pip

4. STL içinde dinamik ardışık koleksiyon hangisidir?

- A) `std::vector`
- B) `std::mutex`
- C) `std::thread`
- D) `std::optional`

5. Kaynakların nesne yaşam süresiyle yönetilmesi hangi prensiptir?

- A) RAII
- B) ODBC
- C) MVC
- D) RTTI

Cevap Anahtarı

1: B | 2: A | 3: B | 4: A | 5: A

Son kontrol listesi

- Kod uyarısız veya açıklanmış uyarılarla derleniyor.
- Sahiplik modeli anlaşılır.
- Raw new/delete kullanımı gereksiz yere yok.
- STL algoritmaları uygun yerlerde tercih edilmiş.
- Testler bağımsız çalışıyor.
- CMake temiz build klasöründe projeyi oluşturabiliyor.